PORTFOLIO 2026

# Portfolio by Minsol Kim

김민솔 | Backend Developer

Kafka 파이프라인, 결제 정합성, 외부 API 안정화

CONTACT

asle14910@gmail.com | github.com/asle149 | https://asle149.github.io/

ABOUT ↗

PROJECTS ↗

CONTACT ↗

ABOUT

# 김민솔 | Backend Developer

## EXPERIENCE

### AI·SW마에스트로 | 17기 연수생 / 백엔드

2026.04 – 2026.11

- 팀장, 백엔드 개발
- 바통: 업무 기록을 검증 가능한 사실 단위인 클레임(Claim)으로 구조화하는 사내 지식 관리 플랫폼
- 클레임 추출, 문서 검사와 확인 질문, 조직 및 열람 권한, 업무 이력과 인수인계 기능 구현

EDUCATION

동국대학교 정보통신공학전공
2021.03 – 2027.02 (졸업 예정)

CERTIFICATE

정보처리기사
SQL개발자(SQLD)
TOEIC Speaking IH(150)
신HSK 4급(253/300)

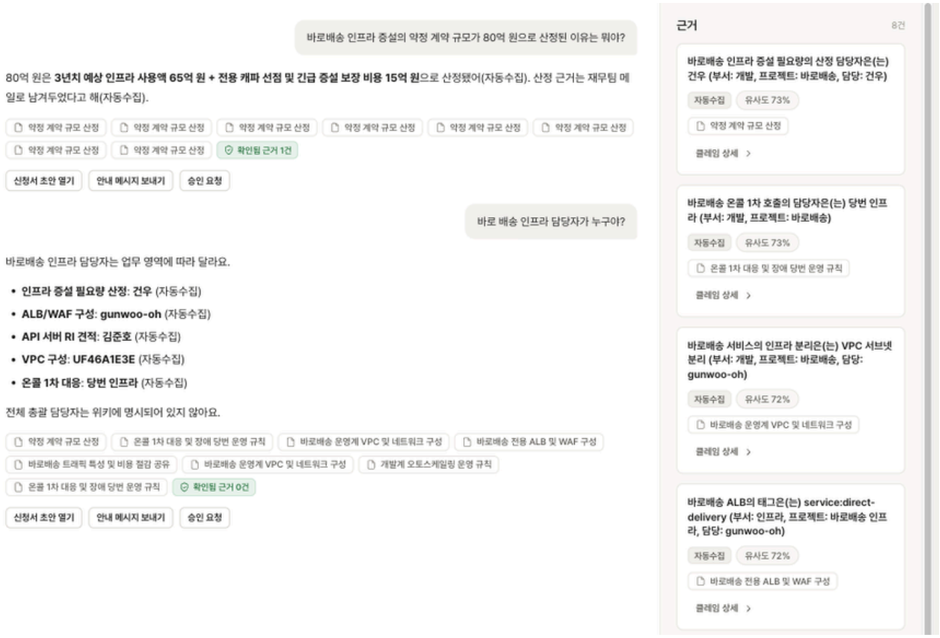
SKILLS

Java, Spring Boot, JPA, Spring Security
Kafka, PostgreSQL, Redis
AWS EC2, Docker

↗

AI·SW마에스트로 17기 / 3인 / 2026.04 – 2026.11

# 바통 (Baton) | 사내 지식 관리 플랫폼




검색 챗봇 화면 (답변의 근거 문장과 원본 출처 표시)

**48%**

문서 984건의 클레임 추출 시간 단축

58.9분 → 30.6분

## 클레임 추출 구현

- 클레임(Claim): 업무 사실을 비교할 수 있도록 구조화한 최소 단위
  주어, 속성, 값과 긍정/부정 여부, 적용 조건, 유효 시점을 함께 저장
- Kafka 컨슈머의 메시지 처리와 LLM 호출 분리
- DB 작업 큐와 워커 풀을 활용한 클레임 추출 작업 병렬 처리
- 조직 및 열람 권한 관리, 업무 이력 조회 기능 구현

## Spring Batch 기반 문서 검사

- Spring Batch로 검사 대상 선정 후 Kafka로 검사 요청 전송
- 같은 주어와 속성의 클레임을 비교해 모순과 변경 이력 구분
- 자동 판정이 어려운 경우 담당자에게 질문하고 답변을 판정 규칙에 반영

## 역할

팀장, 스크럼 마스터 / 클레임 추출과 문서 검사, 업무 이력, 인수인계 등 백엔드 개발

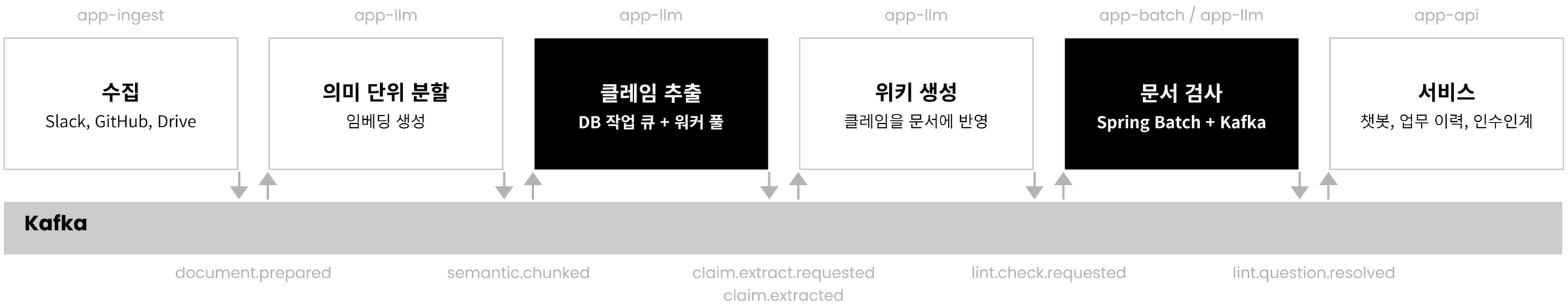
## 기술

Java 21, Spring Boot 3.5, Spring Batch, Kafka, PostgreSQL, Redis, AWS

↗

ARCHITECTURE

# 클레임 추출과 문서 검사 구조




■ 검정 칸 = 직접 구현 (클레임 추출과 문서 검사, 확인 질문)

## ■ 클레임 추출

Kafka 컨슈머는 클레임 추출 작업을 DB에 등록한 뒤 반환
워커 풀은 LLM을 호출해 업무 사실을 클레임으로 구조화
각 클레임에 원문 인용을 연결해 근거 확인

## ■ Spring Batch 기반 문서 검사

Spring Batch로 검사 대상 선정 후 Kafka로 검사 요청 전송
클레임의 값과 적용 조건, 시점을 비교해 모순 여부 판정
자동 판정이 어려운 경우 담당자에게 확인 질문 생성

| | |
|---|---|
| **Transactional Outbox** | 업무 데이터와 전송할 이벤트를 같은 DB 트랜잭션에 저장 |
| **멱등성 보장** | 동일 메시지를 다시 수신해도 처리 결과를 한 번만 반영 |
| **단계별 재시도** | .retry.1m → .retry.10m 순서로 재시도 후 실패가 지속되면 DLQ로 전송 |
| **읽기 전용 DB 뷰** | DB 뷰로 데이터를 조회하고 이벤트로 상태 변경 요청 |

↗

BOTTLENECK

# Kafka 메시지 전달과 클레임 추출 작업 분리

## 문제

- Kafka 컨슈머가 LLM 응답을 기다리는 동안 다음 메시지 처리 지연
- 비동기 호출을 늘려도 배치 전체가 끝날 때까지 대기

## 해결

- Kafka 컨슈머는 작업을 DB에 등록한 뒤 반환하도록 실행 범위 축소
- 워커는 동시 실행 자리가 빌 때마다 DB 큐에서 다음 작업을 가져와 LLM 호출 지속

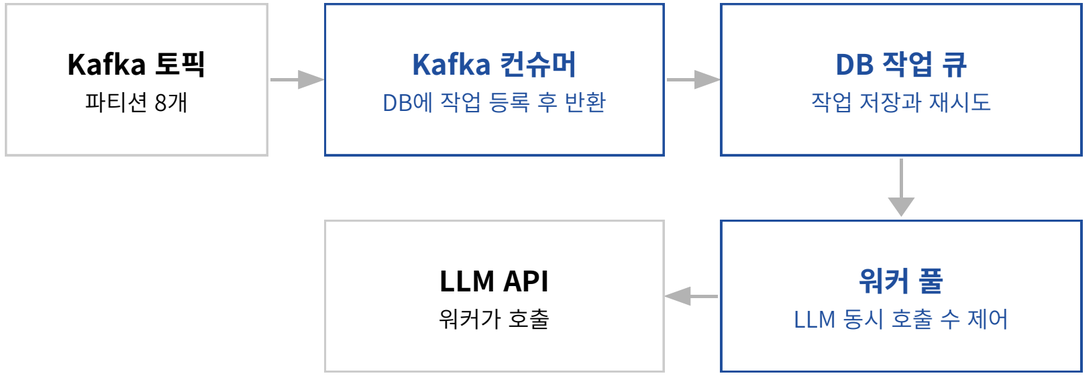
## 검토한 대안

- 파티션 증설: 동시 처리 수는 증가하지만 컨슈머의 LLM 응답 대기는 지속
- Share Group: 처리 중 잠금 만료로 작업이 재전달되어 LLM 중복 호출 발생

### 변경 전: Kafka 컨슈머에서 LLM 호출




### 변경 후: 워커 풀에서 LLM 호출



RESULTS

# 클레임 추출 시간과 결과 검증

왼쪽: 동일 문서 984건으로 구조 변경 전후 비교 / 오른쪽: GitHub PR 33건과 Slack 스레드 67건의 처리 결과

### 작업 큐 도입 전후 클레임 추출 시간 (문서 984건)




**48%**

클레임 추출 시간 단축
동일 문서 984건으로 구조 변경 전후 비교
LLM 호출 수: 3,227회 → 3,524회

원문 인용 검증: 클레임에 연결된 인용 26,817건을 원본과 대조
원문 인용 일치율 99.95%, 동일 작업의 중복 추출 미발생

**57초**

문서 100건 추출 완료

**1,556건**

클레임 추출
위키 228페이지

**25초**

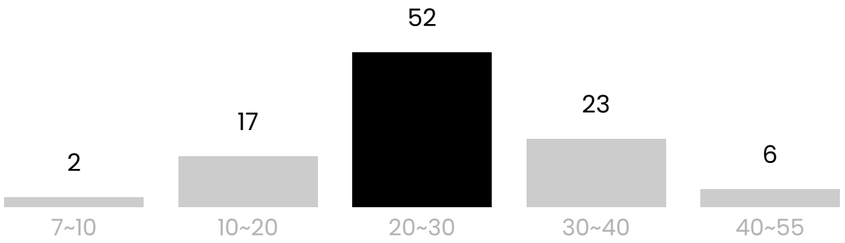
문서당 평균 처리 시간
p95 38초, 최대 53초

**358건/분**

별도 1,024건 추출 실험
문서당 13.4초

문서 100건의 개별 처리 시간 합계: 42분 19초

### 문서당 처리 시간 분포 (초)



# 재시도 토픽과 DLQ 기반 실패 처리




## 동시 호출 제한과 재시도 분산

- Redis에서 서버 전체의 LLM 동시 호출 수를 관리해 공통 상한 적용
- 1분과 10분의 기본 재시도 간격에 지터를 적용해 재시도 요청 집중 방지

## 트러블슈팅

**서킷 브레이커 오작동**

정상 요청도 응답 지연으로 집계되어 후속 호출 차단
→ 지연 판정 기준을 60초에서 300초로 조정

**LLM 응답 잘림**

출력 토큰 한도에 도달해 추출 결과 일부 누락
→ 최대 출력 토큰 수 상향

## 실패 원인별 처리

| | |
|---|---|
| **429 요청 한도 초과** | 재시도 토픽으로 메시지 전송 후 지정된 재시도 시각 이후 재처리 |
| **402 API 잔액 부족** | 서킷 브레이커를 OPEN으로 전환해 추가 LLM 호출과 신규 작업 처리 중단 |
| **일시적 오류 반복** | 서킷 브레이커가 OPEN 상태이면 작업 처리 중단 및 운영 알림 기록 |
| **재시도 횟수 초과** | 메시지를 DLQ로 전송하고 실패 이력에 HTTP 상태 코드와 실패 원인 기록 |

상단: 처리 실패 시 재시도 경로 / 오류 원인에 따라 재시도 또는 추가 호출 중단

↗

BATON / LINT

# Spring Batch 기반 문서 검사와 확인 질문

## 문제

- 같은 대상의 값이 달라도 적용 조건과 시점에 따라 변경 이력일 수 있음
- 표현이 비슷한 서로 다른 대상을 비교하면 모순으로 잘못 판정

## 설계

- Spring Batch로 검사 대상 선정 후 Kafka로 검사 요청 전송
- 주어와 속성이 같은 클레임의 값과 긍정/부정 여부 비교
- 적용 조건과 유효 시점을 확인해 모순과 변경 이력 구분
- 판단이 어려운 경우 담당자에게 질문하고 답변을 판정 규칙에 반영

## 검토한 대안

- 유사도 임계값 상향: 서로 다른 대상의 비교가 남아 오탐이 지속되어 미채택
- 벡터 검색 후보: 동일 대상 여부를 확인할 별칭 매핑이 없어 모순 판정 보류

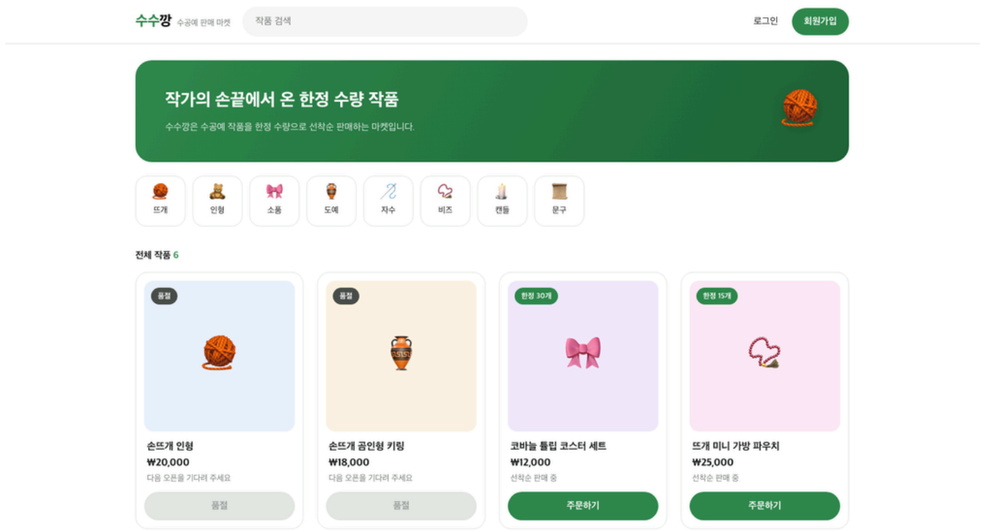



판단이 어려운 클레임은 담당자에게 확인 요청
답변을 판정 규칙에 반영하고 기존 클레임의 변경 이력 유지

↗

SIDE PROJECT / 1인 / 2026.06 – 2026.08

# 수수깡 | 한정 수량 수공예품 판매 플랫폼




작품 마켓 화면 (한정 수량 판매, 테스트 결제 환경)

## 결제 기록 보존과 실패 복구

- 결제사 승인, 취소 및 결과 조회 API 연동
- 결제 요청과 승인 결과를 주문 확정과 별도 트랜잭션으로 저장
- 승인 응답 누락 시 결과 조회, 주문 확정 실패 시 승인된 결제 취소

## 주문 결과에 따른 이벤트 전송과 검증 정책

- 주문 확정 커밋 후 정산 요청, 취소 대기 기록 커밋 후 취소 요청 전송
- 금액과 구매자 검증을 PaymentPolicy로 분리하고 구현체 목록 주입
- 재고 차감과 결제 상태 변경에 조건부 UPDATE를 적용해 중복 처리 방지

## 역할

백엔드 설계, 구현 및 배포

## 기술

Java 21, Spring Boot 3.5, JPA, PostgreSQL, Kafka, OpenFeign, 토스페이먼츠, Docker Compose, k6

↗

PAYMENT

# 주문 실패에도 결제 기록을 남기는 트랜잭션 설계

## 주문 실패 시에도 필요한 승인 기록

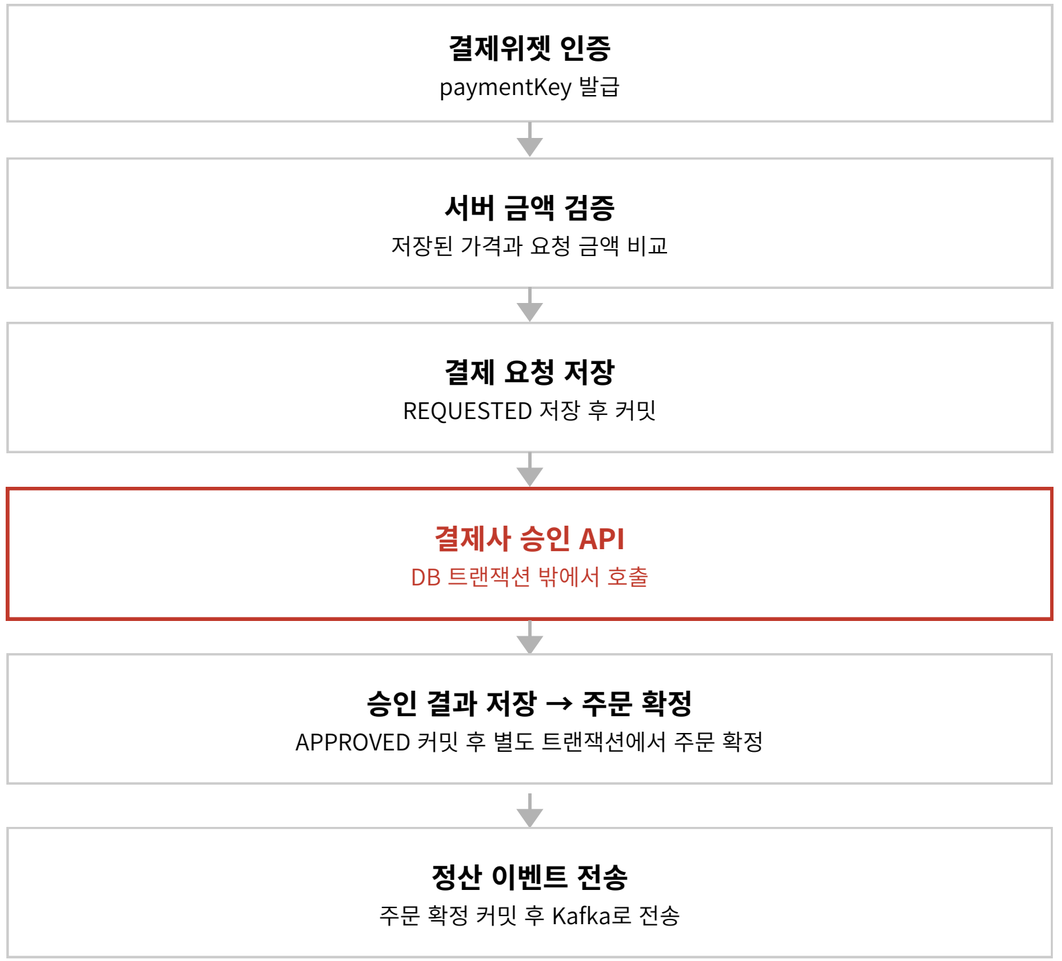
- 주문 예약 만료로 주문 확정에 실패해도 이미 승인된 결제는 취소 필요
- DB 롤백으로 외부 승인을 되돌릴 수 없어 취소할 결제의 기록 보존 필요
- 결제 기록과 주문 변경을 한 트랜잭션으로 묶으면 승인 기록까지 롤백

## 단계별 트랜잭션 분리

- 승인 API 호출 전에 REQUESTED 저장 후 커밋
- 승인 응답 대기 중 DB 커넥션을 점유하지 않도록 API는 트랜잭션 밖에서 호출
- APPROVED 저장과 주문 확정을 별도 트랜잭션으로 처리
- 주문 확정 실패 후 별도 트랜잭션에서 CANCEL_PENDING 저장

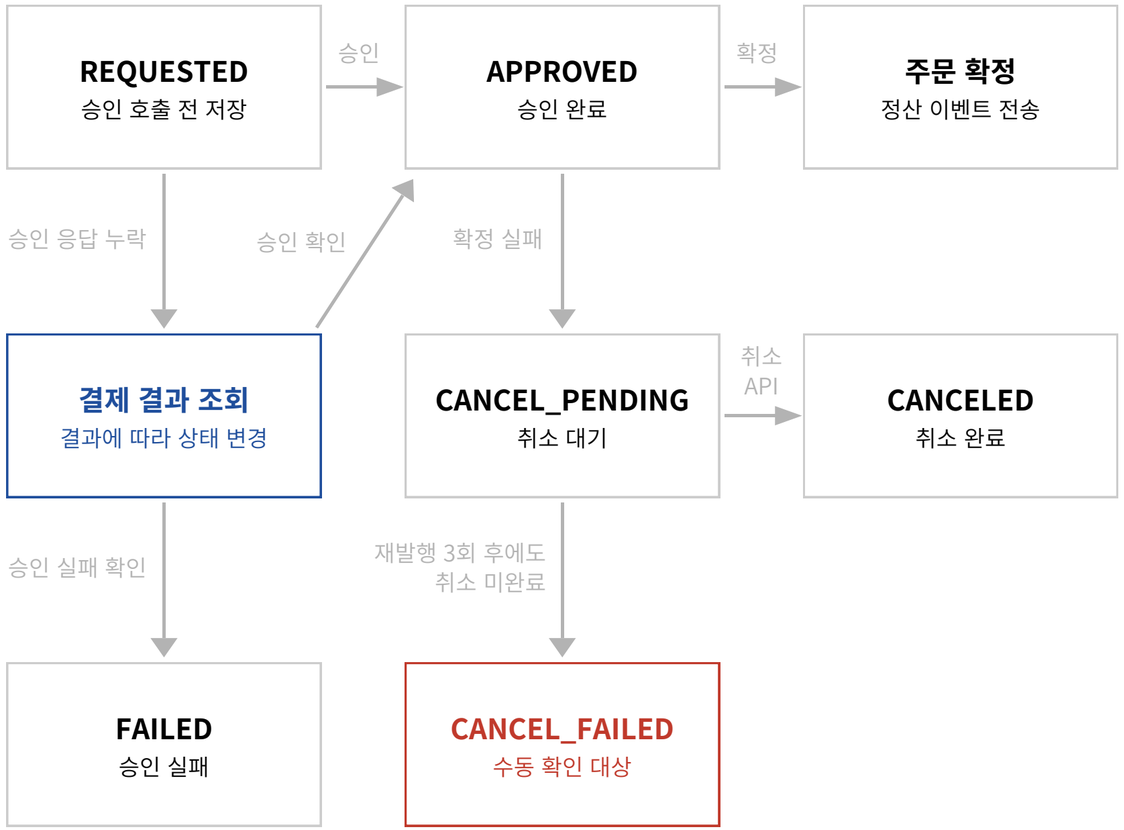
## 중단된 결제의 후속 처리

- 승인 응답을 받지 못하면 REQUESTED 기록을 기준으로 결제사 결과 조회
- 취소가 완료되지 않으면 CANCEL_PENDING 기록을 기준으로 취소 재처리



RECOVERY

# 승인 응답 누락과 주문 확정 실패 처리




## 승인 응답을 받지 못한 경우

- 승인 여부가 불명확하므로 승인 API 재호출 없이 결제사 결과 조회
- 조회로도 결과를 확인하지 못하면 사용자에게 '확인 중'으로 응답
- 스케줄러가 미확정 결제를 주기적으로 재조회해 결제사 결과에 따라 상태 갱신

## 승인 후 주문 확정이 실패한 경우

- 주문 확정 실패 후 별도 트랜잭션에서 CANCEL_PENDING 저장
- DB 커밋 후 취소 요청을 Kafka로 전송, 컨슈머에서 취소 API 호출
- 스케줄러에서 취소 미완료 건의 이벤트를 최대 3회 재발행
- 재발행 후에도 취소되지 않으면 CANCEL_FAILED로 변경해 수동 확인 대상으로 분류

↗

SPRING

# 주문 처리 결과에 따른 정산과 취소 요청

## 주문 확정 전에 정산 요청을 보내는 문제

- 주문 확정 트랜잭션 안에서 Kafka로 정산 요청을 보내면
  이후 DB가 롤백되어도 이미 전달된 요청의 취소 불가
- 주문 확정 결과가 DB에 반영된 뒤 후속 요청을 보내도록 전송 시점 분리

## 커밋 후 이벤트 리스너에서 Kafka 전송

- 상태 변경 서비스에서 Spring 이벤트 발행
- @TransactionalEventListener(AFTER_COMMIT)에서 커밋 후 Kafka 전송
- 주문 확정은 정산 요청, CANCEL_PENDING 저장은 취소 요청으로 구분

## 전송 실패 시 보완 범위

- 미완료 취소는 저장된 상태를 조회해 취소 이벤트 재발행
- 정산 이벤트 전송 실패는 로그로 기록하며 자동 재전송은 미구현
- 커밋 후 전송 누락을 보완할 Transactional Outbox 도입 검토

### DB 반영 후 후속 요청 전송




리스너는 요청을 전달하고 Kafka 컨슈머가 후속 처리 실행

SPRING

# IoC를 활용한 결제 검증 정책 분리

## 문제

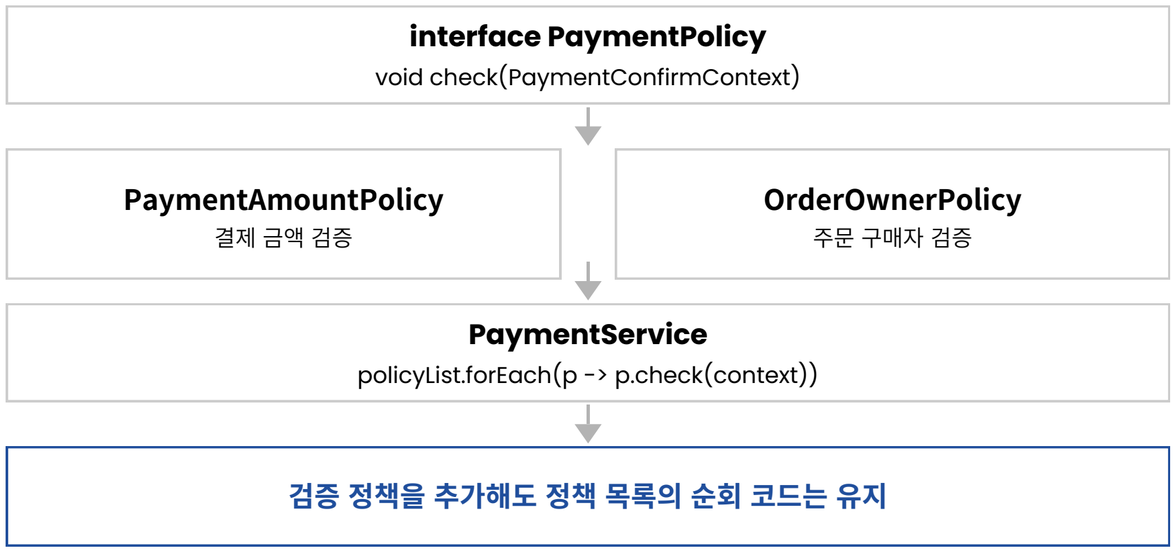
- 결제 서비스 내부에 금액과 구매자 검증 조건문 누적
- 검증 규칙 추가 시마다 결제 서비스의 검증 코드를 수정해야 하는 구조

## 설계

- 각 검증 클래스에서 PaymentPolicy의 check() 구현
- Spring이 등록된 구현체를 List<PaymentPolicy>로 주입
- PaymentService에서 주입받은 목록을 순회하며 check() 호출

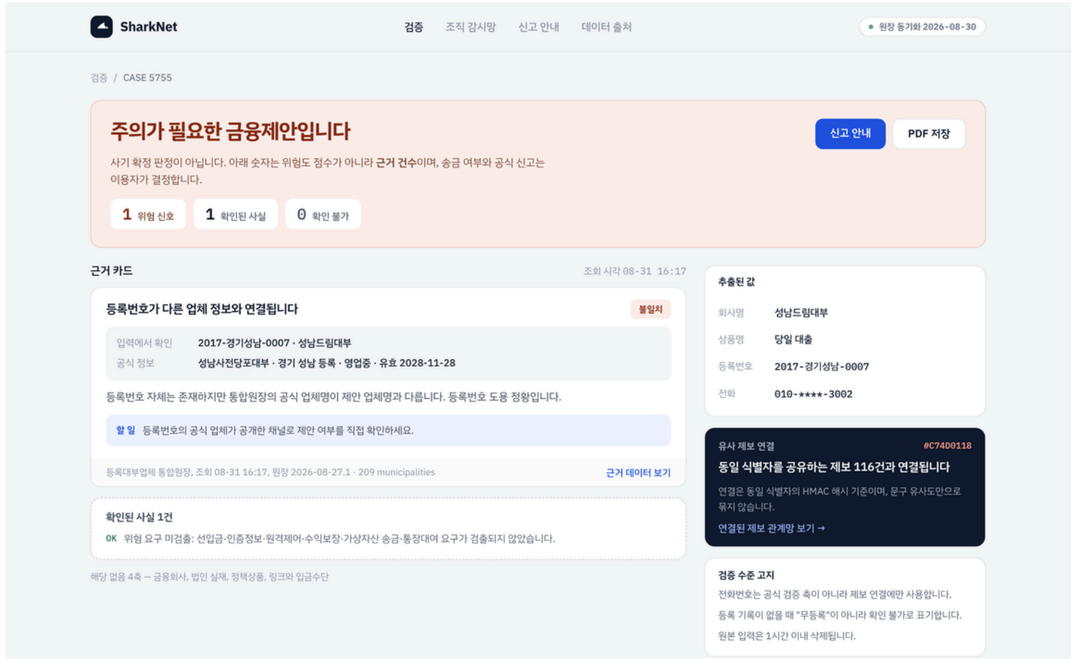
## 구매자 검증 추가 시 변경 내용

- OrderOwnerPolicy 추가 및 검증 컨텍스트에 구매자 정보 전달
- 기존 정책 목록의 순회 코드 변경 없이 구매자 검증 추가



2026 금융 AI CHALLENGE / 2인 / 2026.07 – 2026.09

# 샤크넷 (SharkNet) | 금융 권유 검증 서비스




리포트 화면 (공식 데이터와 비교한 결과 및 출처 표시)

## PromQL 모니터링 구현

- HTTP 정상 응답에도 발생하는 외부 자료 대조 실패를 지표로 수집
- 확인 불가 비율과 처리 지연을 계산하는 PromQL 작성
- Grafana 대시보드와 알림 규칙 구현

## Alertmanager 알림 전달 구현

- Alertmanager에서 알림을 그룹화해 알림 수신 서버로 전달
- 수신 서버에서 Slack 알림 전송과 Android 앱용 조회 API 제공
- 알림 전송 실패 시 Outbox에 저장된 알림 재전송

## 역할

팀장 / 모니터링, 알림 수신 서버와 배포 환경 구축 등 백엔드 개발

## 기술

Java 21, Spring Boot, PostgreSQL, Redis Streams, Redis Pub/Sub
Prometheus, PromQL, Alertmanager, Grafana, Docker

↗

SCALE-OUT

# 서버 확장 시 SSE 전달과 중복 실행 방지

## 문제

- 서버 메모리에만 보관된 SSE 구독 정보로 인해 다른 서버의 상태 변경 수신 누락
- 서버마다 스케줄러와 Outbox 전송이 실행되어 동일 작업 중복 처리

## 해결

- Redis Pub/Sub으로 서버 간 이벤트 공유 후 SSE로 브라우저에 전송
- 이벤트 수신 누락에 대비해 15초마다 DB 최신 상태를 전송하고 화면 갱신
- PostgreSQL advisory lock으로 스케줄러 중복 실행 방지
- Outbox의 각 알림을 한 서버만 선점하도록 처리해 중복 전송 방지

## 검토한 대안

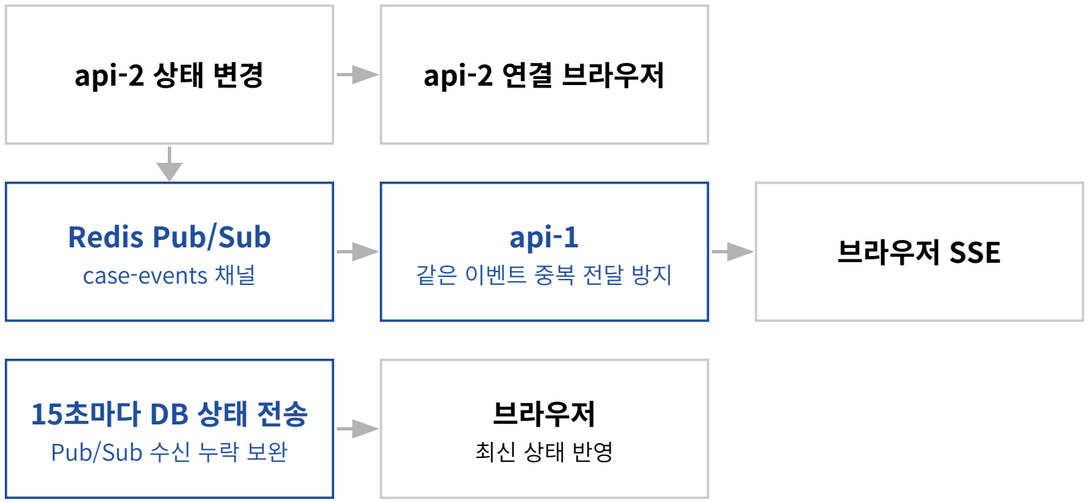
- ShedLock: 별도 테이블이 필요해 기존 PostgreSQL의 advisory lock 선택
- Redis Pub/Sub 단독 사용: 메시지를 보관하지 않아 DB 재조회로 수신 누락 보완

| | |
|---|---|
| **서버 간 SSE 전달** | API 서버 2대에서 8건 모두 수신 (0.66~1.08초) |
| **알림 중복 전송** | 서버 2대가 40건 분담 처리, 중복 전송 0건 |

## 변경 전: 서버별 메모리에 브라우저 연결 보관




## 변경 후: Redis Pub/Sub으로 서버 간 상태 전달




Redis Pub/Sub 차단 실험: DB 재전송 적용 후 8건 모두 화면 갱신

↗

MONITORING

# PromQL 이상 감지와 Alertmanager 알림 전달




## 감시 지표와 목적

- 확인 불가 비율을 기준으로 외부 자료 대조 실패 증가 감지
- Redis Streams 대기 시간을 기준으로 워커의 작업 적체 감지
- 단계별 p95 비교를 통해 지연이 발생한 처리 구간 확인

## 알림 조건과 검증

- 검사 대상이 아닌 NOT_APPLICABLE을 UNKNOWN 비율 계산에서 제외
- 검사 항목 20건 이상, UNKNOWN 비율 50% 초과가 10분간 지속되면 알림 발생
- promtool로 규칙 검증, Slack 전송 오류 해소 후 알림 12건 전달 확인

김민솔 | Backend Developer
asle14910@gmail.com
github.com/asle149

# CONTACT

